Fault Coverage Enhancement via Weighted Random Pattern Generation in BIST using a DNN-driven-PSO Approach

Hillol Maity*, Kaushik Khatua*, Santanu Chattopadhyay*, Indranil Sengupta*,
Girish Patankar† and Parthajit Bhattacharya†
*Indian Institute of Technology, Kharagpur, †Synopsys Inc., India
Email: hillolmaity@gmail.com, kaushik.khatua31@gmail.com, santanu@ece.iitkgp.ac.in, isg@cse.iitkgp.ernet.in,
girish.patankar@synopsys.com, parthajit.bhattacharya@synopsys.com

[1] ***Abstract*—Conventional pseudo-random testing in Built-In-Self-Test (BIST) usually requires a huge amount of testing time. This issue can be addressed with a Weighted Random Pattern generation that can produce test patterns in order to achieve high fault coverage with the fewer number of test vectors. Determining such input weights for a particular circuit is an NP-hard problem. In this paper, we have proposed a technique to converge to a high-quality input weight vector using a Particle Swarm Optimizer (PSO) with the help of a Deep Neural Network (DNN). The DNN prediction of fault coverage value as well as the parallel training of the DNN along with the evolution of the PSO makes this significantly fast. The technique has been tested with ISCAS'85 and ISCAS'89 benchmark circuits. The result shows that the DNN gets capable of predicting the fault coverage values accurately for weight assignments suggested by the particles in PSO. Also, it is observed that the proposed approach is very efficient in covering a large number of faults with less test vectors in self-testing circuits.**

***Index Terms*—BIST, PSO, DNN, Fault Coverage, PRPG**

I. Introduction

Growing circuit complexity of modern integrated circuits (ICs) demands more sophisticated Automatic Test Equipment (ATE) which has a fast-rising cost. Also, with complex embedded System-on-chip (SoC) design, the total number of nodes increases substantially which makes the job for ATE more difficult as it has to scan through all the nodes for faults. The set of test patterns, provided externally by the ATE, requires to cover most of the faults and thus the pattern set becomes too large to be afforded with additional storage elements. Built-In-Self-Test (BIST) is a technique where, an additional hardware feature integrated with the Circuit Under Test (CUT) itself, gets the ability to test its own CUT in terms of its parameters, functionality, etc. Thus, testing of larger and more complex circuits depends no more on the costly ATE and also the additional storage requirement becomes nullified. BIST becomes more useful at situations where the CUT has no external pins for taking input. BIST is, therefore, an easier, faster and cheaper way to tackle the testing of complex circuits of the present day.

[1]This work is partially supported by the research project sponsored by the Synopsys Inc., India

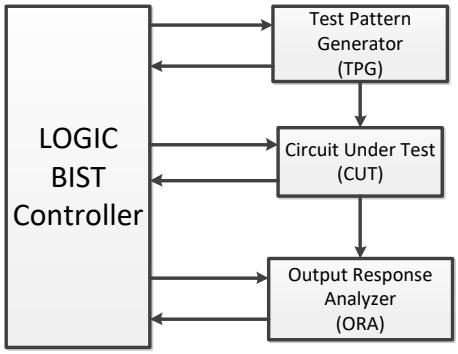

Figure 1: Built-In-Self-Test Basic Structure

Logic BIST (LBIST), which is designed for testing random logic, typically employs a pseudo-random pattern generator (PRPG) to generate input patterns that are applied to the device's internal scan chain. For a particular circuit, the BIST structure is to be designed so that it covers the maximum number of faults in that circuit. Randomly generated test patterns often prove to be an alternative to the deterministic ones as random testing provides cost-effective means of achieving reasonably good fault coverage. However, the set of patterns which will be responsible for covering the majority of the set of faults is dependent on the CUT and the required test length gets too large often. To figure out such a set of faults for a particular circuit is difficult as it will need repeated simulations which itself is time-consuming. To take care of this problem, several algorithms have been proposed in the literature [1] [2] [3]. Among them, Random Weight Assignment is very popular as it does not require memory overhead while achieves high fault coverage [4] [5] [6] [7] [8]. The weight assignment guides the generation of the set of pseudo-random patterns which will be responsible for covering the majority of the faults. Also, in many cases, the random test lengths can be reduced by orders of magnitude, using weighted random patterns.

Determining such input weight assignment or input weight vector for a particular circuit is an NP-hard problem as each

978-1-7281-6052-8/19/$31.00 ©2019 IEEE
DOI 10.1109/ICIT48102.2019.00047

of the weights can hold any fractional value from '0' to '1' indicating the probability of corresponding bit becoming '1'. In this paper, we have proposed a technique of determining a good input weight vector for a given CUT. The approach incorporates a particle swarm optimization (PSO) technique to search through the possible weight vectors. Even a PSO-based search requires repeated fault simulations of the circuit for the set of patterns generated by each of the input weight vectors. Once a good input weight vector is found, the circuit can be tested for faults with a larger set of patterns generated according to that weight assignment. Thus, to avoid the repeated simulations and reduce the PSO convergence time, we have utilized a deep neural network (DNN) to predict the approximate fault coverage corresponding to a particular weight assignment for a constant size of test patterns. As the DNN prediction is much faster than a thorough fault simulation, the PSO will evolve and converge faster. Thus, we have achieved a very fast technique of obtaining a good input weight vector from a given CUT which will, in turn, guide the BIST to test the whole circuit with a larger set of patterns for much better fault coverage.

The rest of the paper has been organized in the following fashion. In Section II, we have discussed the previous works on BIST for improvement of fault coverage values and how PSO or DNN becomes relevant in this work. In Section III, the details about the used DNN, such as training algorithm and network description have been discussed. In Section IV, the implementation of PSO for this work has been described. In Section V, we have shown how to integrate the DNN with the PSO generations parallelly to achieve fast and accurate optimization. Section VI presents the experimental results of our approach have been shown. Section VII concludes the overall approach proposed in the paper and discusses possible improvements and future works.

II. Literature Survey

Formulation of weighted random testing in BIST for random pattern resistant faults were done on the basis of design effort, hardware overhead, fault coverage, and test length [9]. Also, generating the weights from a set of pre-computed patterns were done rather than fault coverage analysis of the complete circuit using the concept of pattern convergence alone where the reasonable trade-off between test time and hardware cost was maintained [10]. These works show that formulating weights for generating random pattern can be utilized to guide the pattern generation by BIST with the goal to achieve higher fault coverage.

Use of machine learning for testing of digital circuits has proved to work out quite well. One of the uses of machine learning model on BIST is, the pseudo-random patterns are used to extract features from the pattern set and train a network with it so that the network can predict pattern set which will target the random pattern resistant faults [11].

In order to reach an optimal solution for NP-hard problems, various meta-heuristic search algorithms are utilized. Particle swarm optimization (PSO) technique is one of them and has been engaged to solve many NP-hard problems [12]. In a swarm, the particles or the individuals share information among themselves and thus traverse the solution space or search space [13] [14]. Each of the particles moves through the search space based on their own experience as well as the experience acquired by the neighbor particles. Movement of the particle is influenced by the three values-

Inertia- the present value or position

Personal Best- the best position or value the particle has experienced so far

Global Best- the best position or value among a particular generation

The main idea is to update the particles of the previous generation with the influence of global best, personal best and inertia values. The modeling of such social behavior is a search process in which particles move toward the best value or position which is also called as the best particle.

III. DNN for Prediction

Calculation of the exact fault coverage value obtained by simulating the circuit with a set of patterns generated by a particular input weight vector is time-consuming. This calculation is our cost function for the PSO. When the circuit complexity increases and the number of iterations or particles are increased in the PSO, the total number of times the above cost function to be computed also increases. Therefore, the objective is to somehow bypass the exact calculation of fault coverage without significant errors. The Deep Neural Network comes to the rescue here. Once the network is trained, it can predict the fitness value or fault coverage for a certain input weight assignment almost instantaneously. This way, the time-consuming cost function calculation is partially replaced with the DNN. This technique proved to be very effective, accurate and consumed significantly less CPU-time.

A. Training the DNN

In order to utilize the fast prediction of the DNN and thus accelerate the fault coverage calculation, the initial neural network needs to be trained with real sample data [15]. For our purpose, the network size of our DNN has a custom number of hidden-layers which can be adjusted according to the requirement and the number of fully connected neurons depends on the number of inputs for which the DNN is being prepared. The sample data, which is various input weight assignments and its corresponding fault coverage values, is generated using the original fault coverage calculation technique and stored in a database. It is important to notice that although it has been the effort to avoid using original fault coverage calculation in order to speed up the whole process, the database generation is necessary for the training of the DNN. A trade-off has to be maintained between feeding enough sample data so that it would not result in an incomplete training of a DNN with erroneous prediction and feeding more than enough sample data which will cause over-fitting of the neural network as well as adds up to more CPU time due the larger database generation time. Only, 4-8% sample data of the total number

of possible inputs are utilized for the training purpose. So, the time taken for the database generation does not slow down the whole process. Out of the total number of database samples, 70% of it are used for actual training purpose, another 15% is used for validation purpose and the rest 15% is used for testing. The validation process is used for tuning hyper-parameters of the DNN. Test data does not affect the network parameters, it is used for measuring the prediction accuracy of the network. The training algorithm used here is the Levenberg-Marquardt (LM) back-propagation model which tries to minimize the mean squared error (MSE) [16]. It is the fastest one among the well known training techniques, although it uses up a high amount of system memory [17]. The training stops whenever the following conditions are met:

1) Maximum number of repetitions (Epochs) is reached
2) Maximum amount of time is exceeded
3) Performance is minimized towards the goal
4) The performance gradient falls below a preset value
5) μ exceeds a preset value
6) Consecutive false validation for more than a preset number of time

The Levenberg-Marquardt model is a hybrid model of Newton's Method and Gradient Descent. When the scalar quantity μ goes low, the algorithm becomes more like Newton's Method and works fast and accurately near low error region [18]. When μ gets higher value, the algorithm acts as a gradient descent with small step size [19]. So, μ starts with a relatively larger value and reduces with each successive iteration. More information about the LM-training methodology can be found here [16].

IV. Particle Swarm Optimization

PSO is utilized in order to perform a meta-heuristic search or optimization. The population is initialized with a set of particles. Each of the particles denotes a particular solution in the search space of the optimization problem at hand. All the particles have a fitness value or cost of their own. Guided by the self and group intelligence as well as the inertia, particles evolve from one generation to the other. PSO has been utilized in various NP-hard problems of engineering [20] [21]. It has proved to be an great tool for optimization. In order to develop a PSO for the current problem, forming a proper structure of particles and developing proper evolution policy of the particles is necessary. A good evolution algorithm does not converge to a solution at local-minima while exploring through the search space. On the other hand, it is also important to explore the search space thoroughly so that the search process does not skip the optimal solution. This trade-off is to be maintained while developing the evolution technique. Here, we have discussed how we have formed the particle structure and evolution technique of the PSO for our problem, while maintaining a satisfactory trade-off as mentioned above.

A. Particle Structure

The PSO implemented here has different input weight assignments or input weight vectors as the particles. The length of the weight vector will be equal to the number of inputs of the circuit. For example, for a 9 input circuit, the PSO will have a particle structure like $[0.235, 0.378, 0.951, 0.744, 0.102, 0.856, 0.553, 0.219, 0.912]$. Each of the weight assignment representing the probability of that becoming '1'. So, the 5^{th} bit will have 0.102 probability of becoming '1' or rather higher chances of becoming '0'. Whereas, the 4^{th} bit will have a probability of 0.951 of becoming '1'. Each of such weights will also have a fault coverage value in terms of percentage value e.g, 93.415%. Higher the fault coverage value, better is the solution or particle. Thus, the best particle will have the highest fault coverage value. A particular generation of PSO can have a predefined amount of particles which in our case varies from 50 to 200.

B. Particle Evolution

In traditional PSO, a velocity term is defined, which is modified using the local best and global best particles as well as particle's own inertia value. The defined velocity term is then added with the old particle to obtain a new particle. Using the cost function, the fitness value is then measured for the updated particle and thus the local, global and personal best values are updated accordingly. After a certain number of iterations, the global best value converges to an optimal solution which gives the best fitness values. At d^{th} generation, v_i^d is the velocity of the i^{th} indexed particle, x_i^d is the position of the i^{th} particle, p_i is the personal best value, p_g^d is the global best particle and $\phi()$ is a random positive number generator.

$v_i^d = v_i^d + \phi(p_i^d - x_i^d) + \phi(p_g^d - x_i^d);$

$x_i^d = x_i^d + v_i^d;$

In our case, the basic idea of updating the particle is similar. Each of the particles will be responsible for generating a set of patterns and the number of patterns can be pre-defined. A fault simulator is used for simulating the circuit against those generated patterns. The simulator tells the percentage fault coverage value which is the cost of the corresponding particle. The particles at the next generation will have weight values of the weighted mean of the inertia weight, personal best weight and global best weight of present generation. Suppose a particular particle of a particular generation of the PSO is $[W_i^1, W_i^2, W_i^3, W_i^4, W_i^5]$, the personal best weight vector $[P_i^1, P_i^2, P_i^3, P_i^4, P_i^5]$ and the global best weight of $[G_i^1, G_i^2, G_i^3, G_i^4, G_i^5]$. c_i, c_p and c_g are inertia factor, personal best factor and global best factor respectively. Then, at the next generation i^{th} particle will be updated as -

$[\frac{c_i \times W_i^1 + c_p \times P_i^1 + c_g \times G_i^1}{c_i + c_p + c_g}]$ as 1^{st} weight,

$[\frac{c_i \times W_i^2 + c_p \times P_i^2 + c_g \times G_i^2}{c_i + c_p + c_g}]$ as 2^{nd} weight,

$[\frac{c_i \times W_i^3 + c_p \times P_i^3 + c_g \times G_i^3}{c_i + c_p + c_g}]$ as 3^{rd} weight,

$[\frac{c_i \times W_i^4 + c_p \times P_i^4 + c_g \times G_i^4}{c_i + c_p + c_g}]$ as 4^{th} weight,

$[\frac{c_i \times W_i^5 + c_p \times P_i^5 + c_g \times G_i^5}{c_i + c_p + c_g}]$ as 5^{th} weight.

And this updated particle will again generate a set of patterns which will be used for fault simulation of the same circuit and a fault coverage value will come up which will

be the updated cost of the corresponding updated particle. This process will continue until the global best value does not change for a certain number of generations. The best particle will be the particle the PSO has faced throughout the generations and will be the final optimized result by the PSO.

V. Integration of DNN with PSO

In the fault simulation driven PSO based approach, every particle of each generation will rely on the simulation to calculate its cost or fault coverage value. Hence, individual circuit simulation for each of the input weight vector adds up to a significant CPU-runtime overhead. Thus it takes much longer than expected for the PSO to evolve from one generation to the other. As discussed in Section III, this additive timing overhead due to repeated simulation can be bypassed using a DNN which will be able to predict the approximate fault coverage value directly from the input weight vector instantaneously.

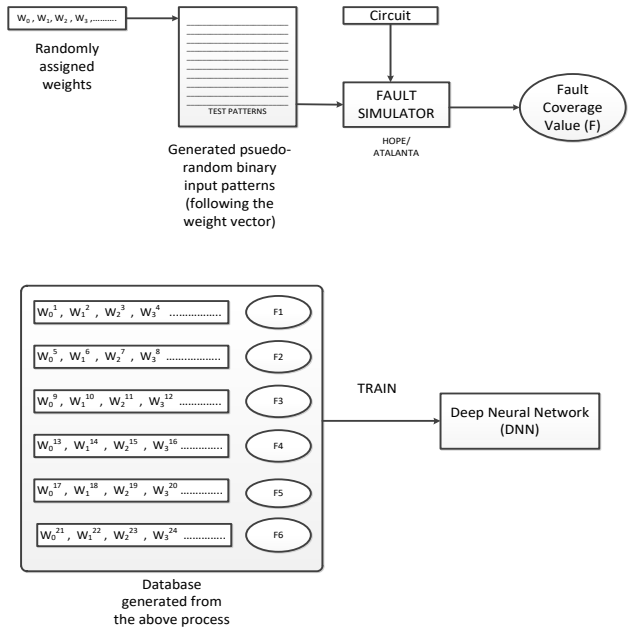

Figure 2: Training of DNN prior to initialization of PSO

Implementing the above-mentioned approach is going to eliminate the timing overheads, such as, time for pseudo-random pattern generation and simulation time of the circuit against those patterns as well. Therefore, at first, a set of data is generated by running the cost function for some initial random input weight vectors. The database is then used for training the prepared DNN. After proper training, the DNN gets capable of predicting the fault coverage values very accurately when compared to the original cost function for fault coverage calculation. The PSO can now completely rely on the trained DNN for evaluating the cost of each particle and evolve accordingly. As the DNN prediction is almost instantaneous, it helps run the PSO at a much higher speed for the rest of the generations. Figure-2 shows the process of how the DNN is

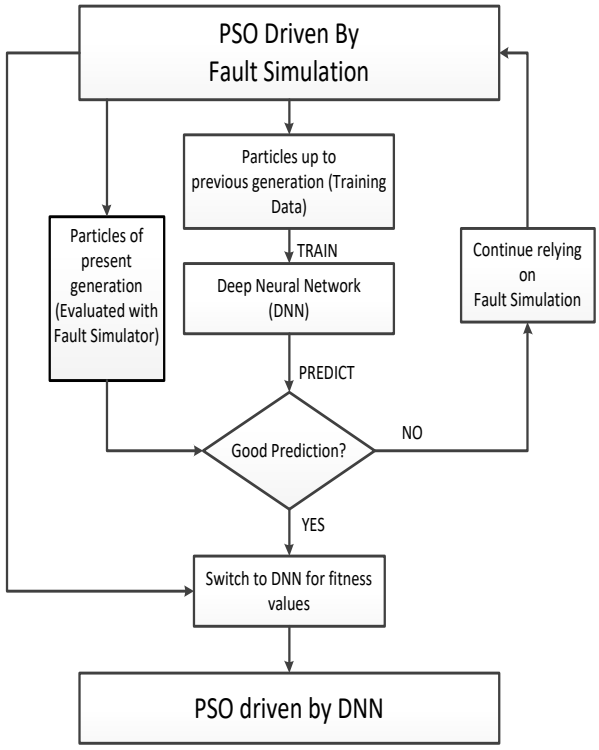

Figure 3: Implementation of DNN driven PSO

trained from the input weight vector. However, this approach also has timing redundancies and scope of improvements. The database that is being generated before training the DNN and before running the PSO as well, has a database generation time which is not small enough to be ignored. The training of the DNN is also a separate process after generation of the DNN and before running the PSO which also takes some time. If the database dimension increases, the network size has to be increased, thus the training increases further. We have taken care of these additional runtimes also by implementing a parallel process as follows-

At the very start, we initialize the PSO with some random particles i.e., input weight vectors. Then we evaluate the cost of each of the particles of the initial generation using the original cost function which is to generate weighted pseudo-random patterns from the given weight vector and then simulate the circuit for faults against those patterns with a fault simulator which will return a percentage fault coverage. Thus, the initial generation will have a complete set of particles with global and personal best values based on the fault coverage value obtained from the fault simulator. Now, this set of particles will be used for training the DNN directly. The PSO will then update its particles and evolve into the next generation. From the second generation onwards, the fault coverage for each of the particles will be evaluated by both the cost function as well as DNN predictor. If the mean error between them is

below a certain pre-defined margin, then the DNN is assumed to have been properly trained and the fault simulator can be completely replaced by the DNN predictor. If the mean error does not get below the pre-defined margin at that generation, the next generation also repeats the same process as that of its previous generation which is an evaluation of cost using the fault simulation and again feeding the new set of data from the present generation for the training of the DNN. By implementing this technique, the database generation time has been eliminated which was being done prior to the PSO run. Also, the training of the DNN will be done in parallel with the evolution of the PSO, this no separate training time needed. The PSO will run almost at the same speed as the fault simulation driven PSO method until the DNN gets well trained. Once, the DNN gets trained, the PSO will switch from the original cost function using fault simulation to the DNN predictor for cost estimation and the PSO being driven by the DNN will start running. The approach is much faster than the fault simulation driven PSO method. The fact that DNN prediction is almost instantaneous while the cost estimation through actually simulating the circuit for a set of patterns takes noticeably longer time, makes the DNN-driven-PSO method much faster. The proposed approach is described in the Figure-3.

VI. Experimental Results

The proposed method has been tested on ISCAS'85 and ISCAS'89 benchmark circuits. The benchmark files have gate-level netlist descriptions of the circuit. Using a fault simulator such as HOPE [22], the circuits were simulated for faults. Both the fault simulator driven PSO and DNN-driven-PSO were tested with several circuits as provided in Table-I. For example, s15850 is a 611 input circuit which has 9780 gates in total. For each of the input weight vectors, 10000 pseudo-random patterns were generated for the purpose of fault simulation and calculation of fault coverage value. The DNN, in this case, had 80-deep layers. Using the fault simulation-driven PSO, the total CPU time it takes to obtain the optimal input weight vector is 32100 seconds and the fault coverage for that is close to 33.842%. The same circuit when goes through the DNN-driven-PSO approach takes about 1035.484 seconds of CPU time while the fault coverage value is about 36.187%. Therefore, the percentage error of the DNN-driven-PSO is about 6.9%. The table shows the circuit description such as the total number of inputs and the total number of gates present in the circuit. As the number of input parameters changed, the hidden layer size of the DNN varied from circuit to circuit. The pattern set size also had to be increased in order to get a decent fault coverage values. The number of inputs of the circuit influences the total number of deep layers to be used whereas the number of gates present in a circuit affected the size of the generated pattern set.

In Table-II, the circuits were simulated against the final input weight vector obtained from the process of Table-I. As the simulation is being done against only one input weight vector, the generated pseudo-random pattern set size was large in order to achieve better fault coverage values unlike the Table-I. For example, the c499 circuit has 41 inputs and 162 gates. It has been tested with 900000 patterns generated according to the input weight vector obtained from the previous experiment where only 10000 patterns were used. It covered 93.098% of faults when tested against the input weight vector obtained from the fault simulation-based PSO. The fault coverage obtained by DNN-driven-PSO method was 95.488%. This table validates that for a larger set of patterns obtained from the input weight given by experiment in Table-I, the DNN-driven-PSO holds its accuracy while being multiple times faster than the simulation based approach.

All the simulation, training and experiment were done on a CPU of the following configuration- Intel i5-4580 @3.2GHz CPU, 4GB DDR3 RAM.

VII. Conclusion

The DNN-driven-PSO has been proved to be a faster way to obtain a good input weight vector in terms of better fault coverage. While being multiple times faster than fault simulation-based PSO in terms of CPU runtime, the accuracy of the DNN-driven-PSO still remains almost intact. Thus, the obtained input weight vector for a particular circuit can help us designing the BIST accordingly. That will allow us to cover the majority of the faults including the random pattern resistant faults. In this work, it was assumed that the BIST is capable of possessing any possible input weight vectors. But it usually has constraints on that and cannot possess any possible input weights. For example, it can have only the weights which are multiples of 0.25 or 0.125, etc. Therefore, this work can be improved by including a method to exclude the weight values that the BIST cannot realize and at the same time, programmable weight sets or probabilities can be considered for maintaining the flexibility of weight assignment which will, in turn, keep the fault coverage intact.

References

[1] S. Wang, "Low hardware overhead scan based 3-weight weighted random bist," in *Proceedings International Test Conference 2001 (Cat. No. 01CH37260)*. IEEE, 2001, pp. 868–877.

[2] X. Zhang, K. Roy, and S. Bhawmik, "Powertest: A tool for energy conscious weighted random pattern testing," in *Proceedings Twelfth International Conference on VLSI Design.(Cat. No. PR00013)*. IEEE, 1999, pp. 416–422.

[3] A. Jas, C. Krishna, and N. A. Touba, "Hybrid bist based on weighted pseudo-random testing: a new test resource partitioning scheme," in *Proceedings 19th IEEE VLSI Test Symposium. VTS 2001*. IEEE, 2001, pp. 2–8.

[4] M. A. Miranda and C. A. López-Barrio, "Generation of optimized single distributions of weights for random built-in self-test," in *Proceedings of IEEE International Test Conference-(ITC)*. IEEE, 1993, pp. 1023–1030.

[5] H.-J. Wunderlich, "Protest: a tool for probabilistic testability analysis," in *Proceedings of the 22nd ACM/IEEE Design Automation Conference*. IEEE Press, 1985, pp. 204–211.

[6] M. Bershteyn, "Calculation of multiple sets of weights for weighted random testing," in *Proceedings of IEEE International Test Conference-(ITC)*. IEEE, 1993, pp. 1031–1040.

[7] J. A. Waicukauski, E. Lindbloom, E. B. Eichelberger, and O. P. Forlenza, "A method for generating weighted random test patterns," *IBM Journal of Research and Development*, vol. 33, no. 2, pp. 149–161, 1989.

[8] R. Kapur, S. Patil, T. J. Snethen, and T. W. Williams, "Design of an efficient weighted random pattern generation system," in *Proceedings., International Test Conference*. IEEE, 1994, pp. 491–500.

Circuit	No. of gates	Size of pattern set	Deeplayers	PSO Time (s)	PSO Fault Coverage (%)	DNN-PSO Time (s)	Predicted Fault Coverage	% Error of fault coverage
s208 (19)	96	1000	20	100.2	80.465	20.156	83.721	+4.04
c432 (36)	120	1000	20	157.288	92.939	48.797	90.076	-3.08
c499 (41)	162	10000	30	1594	97.098	314.411	98.153	+1.08
c880 (60)	320	10000	50	2514.29	77.813	291.227	77.813	+3.38
s1423 (91)	657	10000	50	3361.57	92.344	352.686	96.182	+4.00
c5315 (178)	1726	10000	70	11047.4	89.925	480.322	92.552	+2.92
c7552 (207)	3512	10000	70	13226.6	91.523	551.108	86.121	-5.90
s15850 (611)	9780	10000	80	32100	33.842	1035.484	36.187	+6.9
s13207 (700)	7985	10000	80	22698	37.504	748.514	38.648	+3.05
s38584 (1464)	19335	10000	100	126972	32.992	3023.143	30.014	-9.03
s38417 (1664)	22179	10000	100	128566	34.785	3257.022	38.736	+11.36
s35932 (1763)	16353	10000	100	126794	36.283	2817.644	39.978	+10.18

Table I: Comparison of fault-simulation-driven-PSO approach vs DNN-driven-PSO approach

Circuit	No. of gates	Size of pattern set	Fault coverage given (PSO with fault simulator)	Fault coverage (DNN driven PSO)	% Error of fault coverage
s208 (19)	96	10000	94.884	96.279	+1.47
c432 (36)	120	100000	95.802	93.550	-2.35
c499 (41)	162	900000	93.098	95.488	+2.56
c880 (60)	320	2000000	90.569	87.656	-3.22
s1423 (91)	657	5000000	85.284	88.742	+4.05
c5315 (178)	1726	7000000	90.106	87.313	-3.10
c7552 (207)	3512	7000000	88.956	92.549	+4.00
s15850 (611)	9780	20000000	83.523	87.149	+4.34
s13207 (700)	7985	20000000	85.397	82.606	-2.79
s38584 (1464)	19335	50000000	81.661	79.383	+2.79
s38417 (1664)	22179	50000000	76.298	72.410	-5.00
s35932 (1763)	16535	50000000	71.572	74.675	+4.30

Table II: Comparison of fault coverage values for a larger pattern set obtained PSO and DNN-driven-PSO

[9] J. Hartmann and G. Kemnitz, "How to do weighted random testing for bist?" in *Proceedings of 1993 International Conference on Computer Aided Design (ICCAD)*. IEEE, 1993, pp. 568–571.

[10] F. Brglez, G. Gloster, and G. Kedem, "Built-in self-test with weighted random pattern hardware," in *Proceedings., 1990 IEEE International Conference on Computer Design: VLSI in Computers and Processors*. IEEE, 1990, pp. 161–166.

[11] C. Fagot, P. Girard, and C. Landrault, "On using machine learning for logic bist," in *Proceedings International Test Conference 1997*. IEEE, 1997, pp. 338–346.

[12] R. Eberhart and J. Kennedy, "A new optimizer using particle swarm theory," in *MHS'95. Proceedings of the Sixth International Symposium on Micro Machine and Human Science*. Ieee, 1995, pp. 39–43.

[13] R. Karmakar and S. Chattopadhyay, "Window-based peak power model and particle swarm optimization guided 3-dimensional bin packing for soc test scheduling," *Integration*, vol. 50, pp. 61–73, 2015.

[14] R. Karmakar, A. Agarwal, and S. Chattopadhyay, "Particle swarm optimization guided multi-frequency power-aware system-on-chip test scheduling using window-based peak power model," in *18th International Symposium on VLSI Design and Test*. IEEE, 2014, pp. 1–6.

[15] M. Courbariaux, Y. Bengio, and J.-P. David, "Binaryconnect: Training deep neural networks with binary weights during propagations," in *Advances in neural information processing systems*, 2015, pp. 3123–3131.

[16] J. J. Moré, "The levenberg-marquardt algorithm: implementation and theory," in *Numerical analysis*. Springer, 1978, pp. 105–116.

[17] P. Różycki, J. Kolbusz, and B. Wilamowski, "Dedicated deep neural network architectures and methods for their training," in *2015 IEEE 19th International Conference on Intelligent Engineering Systems (INES)*. IEEE, 2015, pp. 73–78.

[18] R. Battiti, "First-and second-order methods for learning: between steepest descent and newton's method," *Neural computation*, vol. 4, no. 2, pp. 141–166, 1992.

[19] R. Jozefowicz, W. Zaremba, and I. Sutskever, "An empirical exploration of recurrent network architectures," in *International Conference on Machine Learning*, 2015, pp. 2342–2350.

[20] R. Karmakar, A. Agarwal, and S. Chattopadhyay, "Testing of 3d-stacked ics with hard-and soft-dies-a particle swarm optimization based approach," in *2015 IEEE 33rd VLSI Test Symposium (VTS)*. IEEE, 2015, pp. 1–6.

[21] R. Karmakar and S. Chattopadhyay, "Thermal-safe schedule generation for system-on-chip testing," in *2016 29th International Conference on VLSI Design and 2016 15th International Conference on Embedded Systems (VLSID)*. IEEE, 2016, pp. 475–480.

[22] H. K. Lee and D. S. Ha, "Hope: An efficient parallel fault simulator for synchronous sequential circuits," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 15, no. 9, pp. 1048–1058, 1996.